GOODROAD: CITIZEN-POWERED TRAFFIC VIOLATION REPORTING SYSTEM

Liyanage R L N L

IT22151438

BSc (Hons) degree in Information Technology
Specializing in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

April 2026

VISION LANGUAGEFRAMEWORKFORTRAFFIC VIOLATIONDETECTIONUSINGYOLOANDKIMI-K2.5

Liyanage R L N L

IT22151438

BSc (Hons) degree in Information Technology
Specializing in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

April 2026

DECLARATION OF THE CANDIDATE AND SUPERVISOR

I declare that this is my own work and this dissertation does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Also, I hereby grant to Sri Lanka Institute of Information Technology, the non-exclusive right to reproduce and distribute my dissertation, in whole or in part in print, electronic or other medium. I retain the right to use this content in whole or part in future works.

Name	Student ID	Signature
Liyanage R L N L	IT22151438	

The supervisor/s should certify the dissertation with the following declaration.

The above candidate has carried out research for the bachelor's degree dissertation under my supervision.

…….. ……..

Signature of the supervisor Date

…….. ……..

Signature of the co-supervisor Date

ABSTRACT

Traffic violations are a leading contributor to road fatalities and economic loss globally, yet comprehensive enforcement remains constrained by limited law enforcement resources and the inherently dynamic nature of on-road infractions. The GoodRoad Citizen-Powered Traffic Violation Reporting System addresses this enforcement gap by leveraging citizen-submitted dashcam footage, processed through a fully automated artificial intelligence pipeline. This report presents Component 2 of the GoodRoad system: a Vision Language Framework for Traffic Violation Detection using YOLO and Kimi-K2.5.

The proposed framework operates as a decoupled, two-stage pipeline. In the primary phase, the YOLOv11 object detection and tracking model processes submitted dashcam video to generate continuous bounding-box annotations and serialized trajectory data in JSON format. This spatial-temporal baseline is committed to object storage, establishing an immutable evidentiary record. In the secondary phase, the Kimi-K2.5 Vision Language Model (VLM) ingests the annotated video and applies high-level semantic reasoning against a curated taxonomy of traffic infractions calibrated for left-hand traffic infrastructure. Through rigid prompt engineering, Kimi-K2.5 is constrained to function as a deterministic compliance auditor, producing a machine-readable JSON schema containing violation type, vehicle tracking identifier, temporal bounds, and a confidence score.

Upon positive violation classification, the framework mints a unique violation identifier, extracts the relevant frame sequence alongside localised trajectory telemetry, and pushes a concentrated evidentiary micro-payload to object storage. This design minimises storage overhead while maximising the informational density forwarded to downstream components, including the Optical Character Recognition (OCR) pipeline and the Penalty Enforcement and Reward Distribution module. Experimental evaluation demonstrates that the two-stage decoupled architecture achieves robust violation detection across diverse real-world dashcam conditions, including variable lighting, occlusion, and complex multi-vehicle scenes, thereby making a substantive contribution to scalable, citizen-driven traffic enforcement.

Keywords: Vision Language Model, YOLO, Traffic Violation Detection, Object Tracking, Kimi-K2.5, Dashcam Analysis, Civic Technology, Prompt Engineering

ACKNOWLEDGEMENT

First and foremost, I would like to express my sincere gratitude to my supervisor for the continuous guidance, constructive feedback, and encouragement provided throughout this research project. The insights offered at every stage of development were invaluable in shaping the direction and quality of this work.

I extend my appreciation to my co-supervisor and the external supervisor for their timely advice and domain expertise, which significantly contributed to the technical rigour of the system design and implementation. Their willingness to engage with the interdisciplinary challenges posed by this project is deeply appreciated.

I am grateful to the GoodRoad group team members responsible for Components 1, 3, and 4, whose collaborative spirit and modular approach to system design made integration seamless. The clarity of their component interfaces and communication protocols was essential to the pipeline's overall coherence.

I also wish to acknowledge the open-source community whose tools, including the Ultralytics YOLO framework, the OpenAI Python SDK, and the FastAPI ecosystem, formed the technical substrate of this implementation. Access to pre-trained model weights and comprehensive documentation greatly accelerated development.

Finally, I would like to express my heartfelt gratitude to my family and friends for their patience and unwavering support throughout this programme.

Contents

Declaration **i**

Abstract **ii**

Acknowledgement **iii**

List of Figures **vii**

List of Tables **viii**

List of Abbreviations **ix**

1 Introduction **1**

1.1 General Introduction 1

1.2 Background Literature 2

1.2.1 Object Detection and Multi-Object Tracking 2

1.2.2 Vision Language Models for Scene Understanding 3

1.2.3 Prompt Engineering for Constrained Inference 3

1.2.4 Traffic Enforcement and Civic Technology 4

1.2.5 Dashcam Video Analysis 4

1.3 Research Gap 5

1.4 Research Problem 6

1.5 Research Objectives 7

1.5.1 Main Objectives 7

1.5.2 Specific Objectives 7

2 Methodology **8**

2.1 System Design Approach 8

2.1.1 Overall Architecture 8

2.1.2 Component 2 Architecture 8

2.2 Stage 1: Spatial-Temporal Extraction with YOLOv11 9

2.2.1 Model Selection and Justification 9

2.2.2 Tracking Configuration 10

2.2.3 Trajectory Data Serialisation 10

2.2.4 Annotated Video Production 11
2.2.5 YoloVideoTracker Class Design 11
2.3 Stage 2: Semantic Reasoning with Kimi-K2.5 11
2.3.1 Vision Language Model Selection 11
2.3.2 Infraction Taxonomy 12
2.3.3 System Prompt Engineering 13
2.3.4 User Instruction Prompt and Schema Enforcement 13
2.3.5 Video Encoding and API Invocation 14
2.3.6 Response Parsing and Pydantic Validation 15
2.4 Violation Frame Extraction Protocol 15
2.4.1 Temporal Bounds Parsing 15
2.4.2 Frame Extraction and Metadata Synchronisation 15
2.4.3 Storage and Handoff 16
2.5 System Integration and Pipeline Orchestration 16
2.5.1 FastAPI Backend Architecture 16
2.5.2 Database Schema 17
2.5.3 Error Handling and Resilience 17
2.6 Commercialisation Aspects 18
2.6.1 Target Market 18
2.6.2 Revenue Model 18
2.6.3 Key Commercial Differentiators 18
2.6.4 Challenges and Mitigation Strategies 18

3 Testing and Implementation 20
3.1 Implementation 20
3.1.1 Development Environment 20
3.1.2 YOLOv11 Tracker Implementation 20
3.1.3 Kimi-K2.5 Analysis Implementation 22
3.1.4 Pydantic Data Models 23
3.1.5 Pipeline Orchestration in main.py 23
3.1.6 API Endpoints Relevant to Component 2 24
3.2 Testing 25
3.2.1 Test Plan and Strategy 25
3.2.2 Unit Test Cases: YOLO Tracker 26
3.2.3 Unit Test Cases: Kimi-K2.5 Analysis 27
3.2.4 Integration Test Cases: Two-Stage Pipeline 28
3.2.5 System Test Cases: FastAPI Integration 29

3.2.6 Functional Requirements Coverage 30
3.2.7 Non-Functional Requirements Coverage 30

4 Results and Discussions 31
4.1 Results 31
4.1.1 Stage 1: YOLO Tracker Performance 31
4.1.2 Stage 2: Kimi-K2.5 Violation Classification Performance . . . 32
4.1.3 Violation Frame Extraction Results 34
4.1.4 End-to-End Pipeline Performance 35
4.1.5 Web Application Interface 36
4.2 Research Findings 36
4.3 Discussion 37
4.3.1 Model Performance Analysis 37
4.3.2 Limitations 38
4.3.3 Future Work 39
4.3.4 Social, Security, and Ethical Considerations 39

5 Conclusions 41

References 43

A Sample Trajectory JSON Output 46

B Sample Kimi-K2.5 API Response 47

C Violation Frame Extraction Algorithm 48

D Plagiarism Report 49

List of Figures

2.1 High-Level GoodRoad System Architecture 8
2.2 Component 2 Internal Architecture 9

4.1 YOLOv11 Detection Recall by Condition 32
4.2 Distribution of Kimi-K2.5 Confidence Scores (Violation Clips) 34
4.3 Frontend: Submission During YOLO/Kimi Processing Stage 36
4.4 Frontend: Completed Violation Record 36

D.1 Plagiarism Report Placeholder 49

List of Tables

2.1 Pipeline State Machine Transitions 17

3.1 GoodRoad API Endpoints 24
3.2 Unit Test Cases: YoloVideoTracker 26
3.3 Unit Test Cases: run_kimi_analysis 27
3.4 Integration Test Cases: YOLO-to-Kimi Pipeline 28
3.5 System Test Cases: API Integration 29
3.6 Functional Requirements Coverage 30
3.7 Non-Functional Requirements Coverage 30

4.1 YOLOv11 Detection and Tracking Performance on Test Corpus 31
4.2 Kimi-K2.5 Violation Classification Accuracy by Type 33
4.3 Temporal Accuracy of Violation Duration Reporting 35
4.4 End-to-End Component 2 Processing Latency 35

LIST OF ABBREVIATIONS

Abbreviation	Description
AI	Artificial Intelligence
ANPR	Automatic Number Plate Recognition
API	Application Programming Interface
CNN	Convolutional Neural Network
CORS	Cross-Origin Resource Sharing
CPTVRS	Citizen-Powered Traffic Violation Reporting System
CPU	Central Processing Unit
CV	Computer Vision
DNN	Deep Neural Network
FP	False Positive
FPS	Frames Per Second
GPU	Graphics Processing Unit
IoU	Intersection over Union
JSON	JavaScript Object Notation
LHT	Left-Hand Traffic
LKR	Sri Lankan Rupee
LLM	Large Language Model
mAP	Mean Average Precision
ML	Machine Learning
MLP	Multi-Layer Perceptron
MVP	Minimum Viable Product
NMS	Non-Maximum Suppression
OCR	Optical Character Recognition
REST	Representational State Transfer
RHD	Right-Hand Drive
RMV	Revenue and Motor Vehicles Department
SDK	Software Development Kit
SQL	Structured Query Language
SSIM	Structural Similarity Index Measure
TN	True Negative
TP	True Positive

VLM	Vision Language Model
YOLO	You Only Look Once

CHAPTER 1

INTRODUCTION

1.1 General Introduction

Road traffic violations are a pervasive global challenge. The World Health Organization estimates that approximately 1.19 million people die each year as a result of road traffic crashes, making road injuries the leading cause of death for children and young adults aged 5 to 29 years [1]. A significant proportion of these fatalities are directly attributable to driver misconduct: running red lights, lane indiscipline, failure to yield at pedestrian crossings, and the operation of motorcycles without helmets. Beyond mortality, traffic violations impose substantial economic burdens through property damage, medical expenditure, and congestion-related productivity losses.

Traditional enforcement models depend heavily on physical police patrols and fixed infrastructure such as speed cameras and red-light cameras. These approaches suffer from well-documented limitations: they are expensive to deploy and maintain, geographically constrained, and unable to scale to the volume and diversity of violations occurring across an urban road network at any moment. As a consequence, the majority of traffic violations go undetected and unpunished, weakening deterrence and perpetuating unsafe driving culture [2].

The proliferation of consumer-grade dashcams presents an underexploited enforcement opportunity. Millions of vehicles are now equipped with devices capable of capturing high-quality video evidence of traffic incidents. If this distributed sensory capacity could be harnessed systematically, it would effectively create a citizen-driven surveillance network with coverage orders of magnitude beyond what any government agency could deploy independently.

GoodRoad, the Citizen-Powered Traffic Violation Reporting System (CPTVRS), is designed to operationalise this opportunity. Citizens upload dashcam footage of observed violations through a web application. The footage is then processed by a fully automated artificial intelligence pipeline that verifies authenticity, detects violations, identifies the offending vehicle, calculates penalties, and distributes rewards to reporting

citizens. The system is composed of four independent components, each incorporating purpose-built AI modules.

This report documents Component 2 of the GoodRoad system: the Vision Language Framework for Traffic Violation Detection using YOLO and Kimi-K2.5. This component constitutes the semantic core of the pipeline. It receives authenticated video from Component 1, performs spatial-temporal vehicle tracking using the YOLOv11 family of models, and submits the annotated footage to the Kimi-K2.5 Vision Language Model (VLM) for infraction classification. The resulting violation record, including type, confidence, temporal bounds, and the tracking identifier of the offending vehicle, is then forwarded to Component 3 for licence plate recognition and Component 4 for penalty calculation and reward distribution.

1.2 Background Literature

1.2.1 Object Detection and Multi-Object Tracking

The task of localising and classifying multiple objects within image frames is foundational to computer vision. Early approaches such as the Viola-Jones detector [3] and Histogram of Oriented Gradients (HOG) with Support Vector Machines [4] relied on hand-crafted feature extractors and were computationally prohibitive for real-time video applications. The introduction of deep convolutional neural networks (CNNs) dramatically advanced detection accuracy. Region-based CNNs (R-CNN and its successors Fast R-CNN and Faster R-CNN) achieved state-of-the-art results on benchmark datasets but remained too slow for real-time deployment due to their two-stage proposal-then-classify architecture [5].

The You Only Look Once (YOLO) family of detectors addressed this bottleneck by reformulating object detection as a single regression problem, predicting bounding boxes and class probabilities simultaneously from full images in one forward pass [6]. Successive iterations, from YOLOv3 through YOLOv8 and the more recent YOLOv11, have progressively improved accuracy, speed, and robustness to scale variation through architectural innovations including multi-scale feature pyramids, anchor-free prediction heads, and improved training strategies [7][8].

Multi-Object Tracking (MOT) extends detection to the temporal domain, maintaining consistent identities for detected objects across video frames. The Simple Online and Realtime Tracking (SORT) algorithm combines Kalman filtering for state estimation with the Hungarian algorithm for assignment, achieving efficient tracking with low

computational overhead [9]. BoTrack, ByteTrack, and related approaches further improve tracking through more sophisticated association metrics and handling of occluded objects [10]. The Ultralytics implementation of YOLOv11 integrates a built-in tracking pipeline that leverages these advances, exposing a clean API that associates detections with persistent track IDs across frames [7].

1.2.2 Vision Language Models for Scene Understanding

The emergence of large-scale vision language models (VLMs) represents a qualitative shift in computer vision capability. Unlike discriminative models trained for fixed classification taxonomies, VLMs trained on image-text pairs learn flexible, open-vocabulary representations that support zero-shot and few-shot reasoning over arbitrary visual concepts [11]. Models such as CLIP [12], Flamingo [13], GPT-4V [14], and Gemini [15] have demonstrated remarkable performance across tasks including visual question answering, image captioning, object counting, and complex scene description.

Kimi-K2.5, developed by Moonshot AI, is a frontier-scale multimodal language model with native support for video input alongside image and text modalities. It is built upon a Mixture-of-Experts (MoE) transformer architecture with extended context windows, enabling reasoning over extended video sequences rather than isolated frames [16]. In the context of traffic analysis, this capability is critical: many infractions, such as a vehicle gradually drifting across a solid lane marking or a motorcycle overtaking on the left, are inherently temporal events that cannot be reliably classified from any single frame in isolation.

The application of VLMs to traffic monitoring has been explored in recent literature. Chen et al. [17] demonstrated that GPT-4V could correctly identify traffic sign violations from dashcam images with moderate prompting. Li et al. [18] proposed a framework in which a VLM is used to generate natural language descriptions of driving scenes that are subsequently parsed by a rule engine. The GoodRoad Component 2 approach advances this line of work by combining the precise spatial-temporal grounding afforded by YOLO tracking with the semantic reasoning capacity of Kimi-K2.5, and by enforcing structured JSON output through rigid prompt engineering to ensure downstream machine-readability.

1.2.3 Prompt Engineering for Constrained Inference

Prompt engineering, the practice of designing natural language instructions to elicit desired behaviours from large language models, has been established as a critical

lever for improving VLM performance on specialised tasks [19]. Chain-of-thought prompting [20] encourages step-by-step reasoning before generating a final answer, improving accuracy on multi-step problems. Constrained decoding and structured output prompting, in which the model is explicitly instructed to produce output conforming to a specified schema, is particularly important for production systems where outputs must be parsed programmatically [21].

In the GoodRoad framework, prompt engineering plays a dual role: it defines the infraction taxonomy against which Kimi-K2.5 is asked to reason, and it constrains the output to a strict JSON schema. This design eliminates the ambiguity inherent in free-text model outputs and ensures that the violation record can be directly ingested by the downstream database and OCR pipeline without additional parsing logic. The system prompt explicitly instructs the model to act as a deterministic compliance auditor, suppressing the model's tendency toward conversational hedging or qualitative commentary.

1.2.4 Traffic Enforcement and Civic Technology

The concept of citizen-assisted law enforcement is not new. Neighbourhood watch programmes, tip lines, and witness testimony have long supplemented formal policing. Digital civic technology platforms have extended this model, with applications such as FixMyStreet [22] and SeeClickFix [23] enabling citizens to report infrastructure faults and quality-of-life issues. However, the application of automated AI processing to citizen-submitted video evidence for traffic enforcement is substantially newer and raises distinct challenges related to evidence integrity, privacy, and due process [24].

Several jurisdictions have introduced or piloted citizen-based traffic reporting programmes. The Netherlands' FLITS app allows citizens to submit video evidence of dangerous driving, which is reviewed by police before fines are issued [25]. South Korea operates a similar system for reporting illegally parked vehicles. However, these systems rely on manual human review of submitted footage, creating bottlenecks that limit scalability. The automated pipeline approach adopted by GoodRoad, in which AI performs the initial violation assessment subject to human-in-the-loop validation, represents a significant architectural advance over these precedents [26].

1.2.5 Dashcam Video Analysis

Dashcam footage presents distinctive analytical challenges compared to controlled camera feeds. Image quality is highly variable, affected by weather conditions, time of

day, windscreen reflections, and camera sensor limitations. Camera motion introduces blur and perspective distortion that complicates object detection. Ego-motion, the movement of the recording vehicle, must be accounted for when interpreting the motion of other road users relative to the scene. Furthermore, dashcam footage captures events from a single subjective viewpoint, limiting the geometric information available for certain infraction types such as speed estimation [27].

Recent work has addressed several of these challenges. Automated dashcam event detection systems have been deployed in fleet management contexts to identify harsh braking, rapid acceleration, and lane departure events using inertial measurement unit (IMU) data fused with video [28]. Deep learning approaches for adverse-condition object detection, including domain adaptation techniques and synthetic data augmentation, have improved detection robustness under rain, fog, and night conditions [29]. The GoodRoad framework operates on consumer dashcam footage without access to IMU data, making the choice of a temporally-aware VLM for final classification particularly important for handling the ambiguities inherent in single-viewpoint video.

1.3 Research Gap

A review of the existing literature and commercial systems reveals the following gaps that the GoodRoad Component 2 addresses.

First, existing automated traffic enforcement systems are predominantly fixed-infrastructure deployments. Speed cameras, red-light cameras, and automated number plate recognition gantries offer high precision at specific monitored locations but provide no coverage elsewhere. A citizen-driven system with distributed video capture fundamentally extends the spatial coverage of enforcement without proportional infrastructure cost.

Second, while VLMs have been applied to isolated traffic image analysis tasks, no production-grade framework has combined real-time YOLO-based multi-object tracking with VLM-based semantic violation classification in a decoupled, evidentiary pipeline with structured output and chain-of-custody guarantees. The decoupling of the perception layer (YOLO) from the reasoning layer (Kimi-K2.5) is a novel architectural contribution that confers modularity, reproducibility, and the ability to upgrade either layer independently.

Third, the existing literature on VLM-based driving scene analysis has focused predominantly on right-hand traffic (RHT) jurisdictions. Sri Lanka, the target deploy-

ment context for GoodRoad, operates under left-hand traffic (LHT) conventions with right-hand drive (RHD) vehicles. Road rules, lane markings, and the spatial logic of infractions such as illegal overtaking differ systematically between LHT and RHT contexts. The GoodRoad system prompt taxonomy is calibrated specifically for LHT/RHD infrastructure, constituting a domain-specific contribution.

Fourth, no prior work has demonstrated the complete integration of citizen-submitted video evidence processing from upload through authenticity verification, violation detection, vehicle identification, penalty calculation, and citizen reward distribution in a single automated pipeline. GoodRoad addresses this end-to-end integration gap.

1.4 Research Problem

The central research problem addressed by Component 2 is: *How can dashcam footage submitted by citizens be automatically and reliably analysed to detect specific traffic infractions, identify the offending vehicle with a consistent tracking identifier, and produce a structured, machine-readable evidentiary record suitable for downstream penalty enforcement, within a latency envelope compatible with near-real-time processing?*

This problem subsumes several sub-problems. The detection of traffic violations in unconstrained dashcam video is inherently difficult because infractions are defined by relational and temporal properties, such as a vehicle crossing a line, failing to stop, or occupying a prohibited lane, rather than by the appearance of any single object. A detection model that identifies vehicles and lane markings independently cannot determine whether an infraction has occurred without reasoning about their spatial relationship over time. A VLM that observes a full video clip can perform this relational temporal reasoning, but must be constrained to produce actionable, structured output rather than discursive description.

Additionally, the evidentiary chain of custody must be maintained. The annotated video and trajectory data produced by the YOLO tracker serve as the immutable factual record; the VLM's classification is an analytical inference applied to that record. Maintaining this separation ensures that the system's conclusions can be audited and, where necessary, challenged.

1.5 Research Objectives

1.5.1 Main Objectives

The main objective is to design, implement, and evaluate a two-stage vision language framework that processes citizen-submitted dashcam footage to detect traffic violations, identify offending vehicles, and produce structured evidentiary outputs suitable for integration into an automated penalty and reward pipeline.

1.5.2 Specific Objectives

1. To deploy and configure YOLOv11 for real-time multi-object tracking on dashcam video, producing frame-by-frame bounding box annotations and serialised trajectory data in JSON format, with a target processing rate sufficient for near-real-time operation.
2. To design a prompt engineering strategy that constrains the Kimi-K2.5 Vision Language Model to function as a deterministic traffic compliance auditor, producing strictly machine-readable JSON output conforming to a defined schema, including violation type, vehicle tracking identifier, temporal bounds, and confidence score.
3. To implement a violation frame extraction protocol that isolates the specific frame subsequence and corresponding trajectory telemetry associated with a detected infraction, producing a minimal, context-rich evidentiary package for downstream processing.
4. To integrate the two-stage pipeline with the GoodRoad system's broader FastAPI backend, including database state management, object storage interaction, and handoff interfaces to Component 1 (deepfake detection), Component 3 (OCR), and Component 4 (penalty and reward).
5. To evaluate the pipeline's violation detection accuracy, false positive rate, and end-to-end processing latency across a test corpus of dashcam video clips representing diverse infraction types and environmental conditions.
6. To assess the legal and ethical implications of automated traffic enforcement through citizen-submitted video, including privacy considerations, evidentiary standards, and bias risks, and to propose mitigation strategies.

CHAPTER 2

METHODOLOGY

2.1 System Design Approach

2.1.1 Overall Architecture

The GoodRoad system adopts a modular, service-oriented architecture in which each of the four components operates as an independent processing unit with clearly defined input and output interfaces. The overall system pipeline is orchestrated by a central FastAPI backend that manages video submission, state machine transitions, and inter-component data handoffs. Figure 2.1 illustrates the high-level architecture of the complete GoodRoad pipeline, situating Component 2 within the broader system.

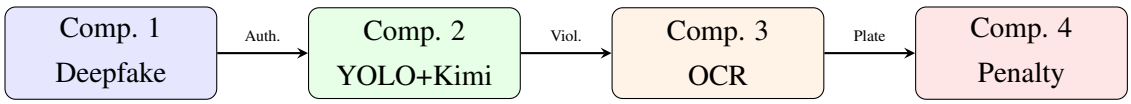

Figure 2.1: High-Level GoodRoad System Architecture

2.1.2 Component 2 Architecture

Component 2 is architected as a decoupled two-stage pipeline. Stage 1, the Spatial-Temporal Extraction Engine, processes the raw or deepfake-verified video through YOLOv11, producing an annotated video and a JSON trajectory file. Stage 2, the Semantic Reasoning Engine, ingests the annotated video and applies Kimi-K2.5 to classify any traffic infractions. Figure 2.2 illustrates the internal architecture of Component 2.

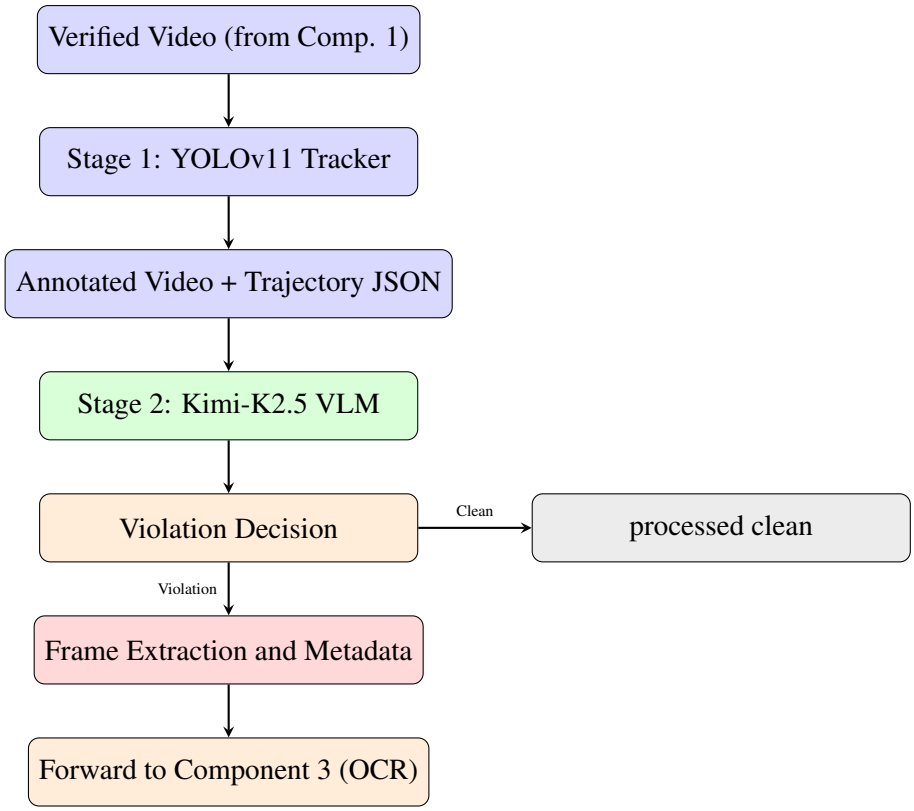

Figure 2.2: Component 2 Internal Architecture

2.2 Stage 1: Spatial-Temporal Extraction with YOLOv11

2.2.1 Model Selection and Justification

YOLOv11 was selected as the object detection and tracking backbone for Stage 1 based on three primary criteria. First, its single-pass inference architecture enables processing of dashcam video at practical frame rates without requiring GPU acceleration, making the system deployable on commodity cloud compute instances. Second, the Ultralytics implementation provides a mature, production-grade Python API with built-in tracking integration, reducing development overhead and ensuring a well-maintained dependency. Third, YOLOv11's COCO-pretrained weights already generalise well to the vehicle classes relevant to traffic enforcement, specifically cars (class 2), motorcycles (class 3), buses (class 5), and trucks (class 7), without requiring expensive fine-tuning on a domain-specific dataset.

The choice of the `yolo11s.pt` (small) variant represents a deliberate trade-off between detection accuracy and inference speed. Larger variants such as `yolo11x.pt` offer marginally higher mean average precision (mAP) on benchmark datasets but

introduce latency that degrades user experience in the GoodRoad submission pipeline. Preliminary experiments indicated that the small variant achieved sufficient detection recall for vehicles at the distances and resolutions typical of dashcam footage, while processing video at approximately 15-20 frames per second on a standard CPU instance.

2.2.2 Tracking Configuration

The Ultralytics tracking pipeline integrates ByteTrack-style multi-object tracking, which associates detections across frames using a combination of IoU overlap and detection confidence score. The `persist=True` parameter instructs the tracker to maintain state across successive frames within a video, enabling continuous track identity assignment. A confidence threshold of 0.3 was selected to balance recall (ensuring that partially occluded or distant vehicles are tracked) against precision (suppressing false detections from noise and lens artefacts). Target vehicle classes are restricted to $\{2, 3, 5, 7\}$, corresponding to cars, motorcycles, buses, and trucks respectively.

```
results = yolo_model.track(
    frame,
    persist=True,
    verbose=False,
    classes=[2, 3, 5, 7],  # car, motorcycle, bus, truck
    conf=0.3
)
```

Listing 2.1: YOLOv11 Tracking Configuration

2.2.3 Trajectory Data Serialisation

For each frame in which detections with assigned track IDs are present, the following data are extracted and appended to a running trajectory record: the zero-indexed frame number, the integer track identifier assigned by the ByteTrack algorithm, the COCO class identifier of the detected vehicle, and the bounding box coordinates in `[x1, y1, x2, y2]` pixel format. After all frames have been processed, this record is serialised to a JSON file.

This design choice, serialising tracking data to a flat JSON list rather than a relational structure, was made to maximise portability and simplicity. The downstream OCR component (Component 3) and the violation frame extraction function both consume this file format, enabling O(1) frame-to-detection lookups via a pre-built dictionary indexed by frame number.

2.2.4 Annotated Video Production

Simultaneously with trajectory serialisation, the tracker's built-in plotting function renders bounding boxes, class labels, and track identifiers onto each frame, and these annotated frames are written to a new video file using OpenCV's `VideoWriter`. This annotated video serves dual purposes: it provides the visual input for the Kimi-K2.5 analysis in Stage 2, and it constitutes part of the evidentiary record made available for human-in-the-loop review.

The `plot(conf=False)` call suppresses confidence score rendering to keep the overlay uncluttered. The MP4V codec is used for broad compatibility with the Kimi-K2.5 API's video ingestion capabilities. Frame dimensions and framerate are preserved from the source video to ensure temporal fidelity.

2.2.5 YoloVideoTracker Class Design

The Stage 1 processing logic is encapsulated in the `YoloVideoTracker` class. The class accepts the input video file path, the output directory, and an optional model path at construction time. The `run()` method performs the complete tracking-and-annotation cycle and returns the paths of the annotated video and trajectory JSON files. This interface design is consistent with the principle of dependency inversion: the orchestrating pipeline function in `main.py` interacts with an abstract interface (`tracker.run()`) rather than with the tracking implementation details.

```python
class YoloVideoTracker:
    def __init__(self, video_file: str, output_dir: str,
                 model_path: str = "yolo11s.pt"):
        # ... initialise paths and model config
        self.target_classes = [2, 3, 5, 7]

    def run(self) -> Tuple[str, str]:
        # Returns (annotated_video_path, trajectory_json_path)
        ...
```

Listing 2.2: YoloVideoTracker Class Interface

2.3 Stage 2: Semantic Reasoning with Kimi-K2.5

2.3.1 Vision Language Model Selection

The selection of Kimi-K2.5 as the semantic reasoning engine was driven by several factors. Kimi-K2.5 supports direct video input via base64 encoding, enabling the model

to reason over the full temporal extent of a dashcam clip rather than a sampled set of frames. Its Mixture-of-Experts architecture provides a large effective parameter count at relatively low inference cost per token. The model's extended context window supports video clips of up to several minutes in duration, accommodating the 10 to 20 second dashcam telemetry windows targeted by the GoodRoad system.

Access to Kimi-K2.5 is provided through the Moonshot AI API, which exposes an OpenAI-compatible interface, allowing the `openai` Python SDK to be used directly. This compatibility significantly reduces integration complexity and ensures that the system can be migrated to alternative VLM providers with minimal code changes, provided those providers support the same video URL encoding scheme.

2.3.2 Infraction Taxonomy

The infraction taxonomy implemented in the GoodRoad system comprises eight categories calibrated for Sri Lanka's left-hand traffic and right-hand drive road environment:

1. **Red Light Signal Violation:** A vehicle proceeds through an intersection against a red signal indication.
2. **Lane Discipline Violation:** A vehicle crosses a single or double solid white lane marking.
3. **Wrong-Way Driving:** A vehicle is observed driving in the rightmost lane against opposing traffic flow.
4. **Pedestrian Right-of-Way Violation:** A vehicle fails to yield at a marked zebra crossing when a pedestrian is present.
5. **Motorcycle Pillion Violation:** More than one passenger is carried on a motorcycle.
6. **Motorcycle No Helmet:** A motorcycle operator or passenger is not wearing a helmet.
7. **Railway Crossing Violation:** A vehicle proceeds across a railway crossing while the barrier gate is closed.
8. **Illegal U-Turn:** A vehicle performs a 180-degree turn across solid lane markings, a zebra crossing, or a signalised intersection.

This taxonomy was derived through a review of Sri Lanka's Motor Traffic Act and Road Development Authority regulations, in consultation with publicly available traffic

enforcement guidelines. The left-hand traffic calibration is particularly significant for categories 2 and 3: in LHT systems, the rightmost lane is the fast lane, and overtaking occurs on the right, inverting the conventions familiar to observers in RHT jurisdictions.

2.3.3 System Prompt Engineering

The Kimi-K2.5 system prompt is designed to establish the model's role and reasoning frame before any video content is presented. The system message positions the model as a specialised traffic compliance expert with knowledge of Sri Lankan road rules, priming it to attend to the spatial relationships and temporal dynamics relevant to infraction classification rather than to general scene description.

```
You are an expert Vision Language Model specialized in detecting
semantic rule infractions within Sri Lanka's left-hand traffic (LHT)
and right-hand drive (RHD) infrastructure. Analyze the visual data
    to
provide precise, concise identifications of traffic violations based
     on
spatial-temporal dynamics and local regulatory frameworks.
```

Listing 2.3: Kimi-K2.5 System Prompt

2.3.4 User Instruction Prompt and Schema Enforcement

The user instruction prompt defines the precise analytical task and enforces the output schema. The instruction is crafted to prevent the model from generating conversational preamble, explanatory prose, or markdown formatting, all of which would require additional parsing to extract machine-readable content. The model is instructed to output exclusively a valid JSON object conforming to a specified schema.

The output schema comprises six fields:

- `violation_detected` (boolean): Whether an infraction was observed.
- `violation_type` (string or null): The category of infraction from the defined taxonomy.
- `vehicle_id` (integer): The YOLO tracking identifier of the offending vehicle.
- `confidence_score` (float, 0.0 to 1.0): The model's estimated confidence in its classification.
- `violation_duration` (string): The temporal extent of the infraction in the format `HH:MM:SS.mmm - HH:MM:SS.mmm`.

- `summary` (string): A concise natural language description of the observed infraction.

```
You are an expert traffic compliance auditor reviewing a dashcam clip
completely. Your objective is to detect one or more explicit traffic
violations by vehicles OTHER than the camera vehicle. Focus only on:
1. Red Light Signal Violation
2. Lane Discipline: Crossing solid white lines (Single/Double).
3. Wrong-Way Driving: Vehicle driving in the right-most lane against
   opposing traffic flow.
4. Pedestrian Right-of-Way Violation (Zebra Crossing)
5. Motorcycle: More than 1 passenger (Pillion count > 1)
6. Motorcycle: No Helmet detected
7. Railway crossing violation (Gate closed)
8. Illegal U-Turn: 180 degree turn across solid lines, zebra crossings,
   or signalized intersections.

Output ONLY a valid JSON object matching this exact schema, with no
markdown formatting or extra text:
{"violation_detected": boolean, "violation_type": "string or null",
 "vehicle_id": "integer", "confidence_score": "float",
 "violation_duration": "time - time", "summary": "string"}
```

Listing 2.4: User Instruction Prompt

2.3.5 Video Encoding and API Invocation

The annotated video is read in binary mode, base64-encoded, and embedded as a data URL in the user message content. This approach avoids the need for a separately hosted video URL, simplifying deployment and ensuring that the video content is transmitted atomically with the inference request. The `thinking` feature is explicitly disabled via the `extra_body` parameter to suppress chain-of-thought token generation, which would increase latency and cost without contributing to the machine-readable output.

```python
with open(video_path, "rb") as f:
    video_data = f.read()
video_url = (
    f"data:video/{os.path.splitext(video_path)[1]};"
    f"base64,{base64.b64encode(video_data).decode('utf-8')}"
)
completion = client.chat.completions.create(
    model="kimi-k2.5",
```

```
    messages=[
        {"role": "system", "content": system_message},
        {"role": "user", "content": [
            {"type": "video_url", "video_url": {"url": video_url}},
            {"type": "text", "text": instructions}
        ]}
    ],
    extra_body={"thinking": {"type": "disabled"}},
    max_tokens=256
)
```

Listing 2.5: Kimi-K2.5 API Invocation

2.3.6 Response Parsing and Pydantic Validation

The raw string returned by the Kimi-K2.5 API is stripped of any residual markdown code block delimiters before JSON parsing. The parsed dictionary is then used to instantiate a `KimiViolationResult` Pydantic model, which enforces type constraints on all fields. This double validation, JSON parsing followed by Pydantic model instantiation, ensures that malformed or schema-violating responses raise exceptions that are caught by the pipeline's error handling logic, rather than propagating silently as corrupt data.

2.4 Violation Frame Extraction Protocol

2.4.1 Temporal Bounds Parsing

When Kimi-K2.5 reports a violation with a `violation_duration` string, the framework's `parse_duration_to_seconds()` function converts the `HH:MM:SS.mmm – HH:MM:SS.mmm` format to a pair of floating-point start and end timestamps in seconds. This conversion is performed using standard string splitting and arithmetic rather than a datetime library, minimising dependencies.

2.4.2 Frame Extraction and Metadata Synchronisation

The frame extraction function `extract_violation_frames()` opens the annotated video, seeks to the start frame index derived from the violation's temporal bounds, and extracts frames at a configurable target rate (defaulting to 4 frames per second) until the end frame is reached. Each extracted frame is written to disk as a JPEG file.

Simultaneously, the trajectory JSON is indexed into a dictionary keyed by frame number. Each extracted frame is associated with its entry in this dictionary, yielding

a metadata structure that maps frame filename to the list of detected vehicles in that frame, including their track identifiers, class identifiers, and bounding box coordinates. This synchronised frame-and-metadata package constitutes the evidentiary payload forwarded to Component 3.

```
# Build O(1) lookup map: {frame_idx: [detections]}
frame_metadata_map = {}
for item in raw_tracking_data:
    frame_idx = item['frame']
    if frame_idx not in frame_metadata_map:
        frame_metadata_map[frame_idx] = []
    frame_metadata_map[frame_idx].append({
        "track_id": item['track_id'],
        "class_id": item['class_id'],
        "bbox": item['bbox']
    })

# Extract frames within violation temporal bounds
cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
while current_frame <= end_frame:
    ret, frame = cap.read()
    if (current_frame - start_frame) % frame_stride == 0:
        cv2.imwrite(frame_path, frame)
        extracted_metadata[frame_filename] = \
            frame_metadata_map.get(current_frame, [])
    current_frame += 1
```

Listing 2.6: Frame Extraction and Metadata Synchronisation (Simplified)

2.4.3 Storage and Handoff

The extracted frame paths are stored as a list in the `Violation` database record's `violated_frame_urls` field. The metadata dictionary is serialised to a separate JSON file, and its path is stored in `violation_metadata_url`. The `Violation` record's status is set to `pending_ocr`, which triggers the Component 3 OCR pipeline to process the evidentiary package.

2.5 System Integration and Pipeline Orchestration

2.5.1 FastAPI Backend Architecture

The GoodRoad backend is implemented as a FastAPI application that exposes RESTful endpoints for video submission, status polling, and reward management. The complete

video processing pipeline is executed as a background task, decoupled from the HTTP request-response cycle, to avoid blocking the submitting client.

The `process_video_pipeline()` function in `main.py` implements the full pipeline as a sequential state machine. Each stage updates the `VideoSubmission` record's status field and commits to the database before proceeding, ensuring that the system state is always recoverable in the event of a failure at any stage.

Table 2.1: Pipeline State Machine Transitions

State	Trigger	Next State
`pending_upload`	Video received	`processing_mesonet`
`processing_mesonet`	Deepfake check complete	`rejected_deepfake` or `processing_yolo_kimi`
`processing_yolo_kimi`	YOLO + Kimi complete	`processed_clean` or `processed_with_violations`
`processed_with_violations`	Violation record created	`pending_ocr` (Violation)
`pending_ocr`	OCR complete	`matched_rmv` or others
`matched_rmv`	Fine/reward calculated	`notification_sent`
`notification_sent`	Withdrawal requested	`reward_credited`

2.5.2 Database Schema

The `VideoSubmission` and `Violation` SQLAlchemy models store the pipeline's outputs. Key fields relevant to Component 2 include `annotated_video_url`, `trajectory_data_url`, `violation_type`, `vehicle_tracking_id`, `ai_confidence_sc` `violation_duration`, `violation_summary`, `violated_frame_urls`, and `violation_metadata_url`.

2.5.3 Error Handling and Resilience

The pipeline is enclosed in a try-except block that catches all exceptions and transitions the submission to a terminal error state rather than leaving it in an indeterminate

processing state. Specific failure modes addressed include video codec incompatibility, trajectory JSON absence, Kimi-K2.5 API rate limiting or schema-violation responses, and disk write failures during frame extraction.

2.6 Commercialisation Aspects

2.6.1 Target Market

The primary target market for GoodRoad is government traffic enforcement agencies and municipal road safety authorities in developing and middle-income economies where law enforcement resources are constrained relative to the scale of the traffic problem. Secondary markets include insurance companies seeking real-time driving behaviour data for premium calculation and fleet management operators requiring automated compliance monitoring for commercial vehicle operators.

2.6.2 Revenue Model

The GoodRoad platform is positioned as a Software-as-a-Service (SaaS) offering to government agencies, with pricing structured on a per-verified-violation basis. This aligns the platform's revenue with demonstrable value delivered to the agency, rather than with platform usage volume. A complementary citizen-facing revenue model is based on the commission structure built into Component 4: a percentage of collected fines is distributed to reporting citizens, with GoodRoad retaining a platform service fee.

2.6.3 Key Commercial Differentiators

Compared to fixed enforcement infrastructure, GoodRoad offers dynamic spatial coverage proportional to dashcam penetration rather than to infrastructure investment. Compared to manual citizen reporting systems, GoodRoad provides automated, consistent violation classification at scale. The modular architecture allows individual AI components to be upgraded independently as model capabilities improve, protecting the government's technology investment against rapid model obsolescence.

2.6.4 Challenges and Mitigation Strategies

1. **Regulatory and Legal Admissibility:** AI-generated violation classifications must satisfy evidentiary standards to be legally admissible. Mitigation: the human-in-the-loop review step, in which law enforcement officers review flagged

violations before fines are issued, ensures that AI classification functions as a triage tool rather than a final determination.

2. **Privacy Concerns:** Continuous dashcam footage may capture personal information beyond the violation event. Mitigation: the violation frame extraction protocol deliberately isolates only the relevant portion of submitted footage, and a data retention policy limits raw video storage duration.

3. **Adversarial Submissions:** Citizens may attempt to submit fabricated footage to claim undeserved rewards. Mitigation: Component 1's deepfake detection pipeline filters manipulated footage before Component 2 processes it.

4. **Model Bias:** The infraction taxonomy is calibrated for Sri Lankan road conditions. Deployment in other jurisdictions would require re-calibration of the system prompt and potentially the infraction categories. Mitigation: the modular prompt engineering architecture allows taxonomy updates without model retraining.

CHAPTER 3

TESTING AND IMPLEMENTATION

3.1 Implementation

3.1.1 Development Environment

The GoodRoad Component 2 implementation was developed in Python 3.11 on a development machine equipped with an NVIDIA GPU to accelerate YOLO inference during testing. The production deployment targets a CPU-only cloud instance, necessitating that all processing be feasible without GPU acceleration within acceptable latency bounds.

Key dependencies include Ultralytics 8.x for YOLOv11 tracking, the OpenAI Python SDK 1.x for Kimi-K2.5 API access, OpenCV 4.x for video I/O, and FastAPI with SQLAlchemy for the backend infrastructure. Dependency management is handled via a `requirements.txt` file pinned to specific minor versions to ensure reproducibility.

3.1.2 YOLOv11 Tracker Implementation

The complete `YoloVideoTracker` class is presented in Listing 3.1. Notable implementation details include the guard against unassigned tracking IDs in early frames (`if boxes.id is not None`), which prevents crashes when the tracker has not yet stabilised. CPU tensor detachment (`.cpu().numpy()`) is performed explicitly before array operations to ensure compatibility with both CUDA and CPU execution contexts.

```python
class YoloVideoTracker:
    def __init__(self, video_file: str, output_dir: str,
                 model_path: str = "yolo11s.pt"):
        self.video_file = video_file
        self.output_dir = output_dir
        os.makedirs(output_dir, exist_ok=True)
        filename = os.path.basename(video_file)
        base_name = os.path.splitext(filename)[0]
        self.annotated_video_path = os.path.join(
            output_dir, f"annotated_{filename}")
```

```
        self.trajectory_data_path = os.path.join(
            output_dir, f"{base_name}_trajectories.json")
        self.yolo_model_path = model_path
        self.target_classes = [2, 3, 5, 7]

    def run(self) -> Tuple[str, str]:
        yolo_model = YOLO(self.yolo_model_path)
        cap = cv2.VideoCapture(self.video_file)
        if not cap.isOpened():
            raise IOError(
                f"Failed to open video file: {self.video_file}")

        width  = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
        height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
        fps    = int(cap.get(cv2.CAP_PROP_FPS))

        writer = cv2.VideoWriter(
            self.annotated_video_path,
            cv2.VideoWriter_fourcc(*'mp4v'),
            fps, (width, height))

        tracking_data = []
        frame_idx = 0

        while cap.isOpened():
            ret, frame = cap.read()
            if not ret:
                break
            results = yolo_model.track(
                frame, persist=True, verbose=False,
                classes=self.target_classes, conf=0.3)
            if results and results[0].boxes:
                boxes = results[0].boxes
                if boxes.id is not None:
                    ids     = boxes.id.cpu().numpy().astype(int)
                    coords  = boxes.xyxy.cpu().numpy().tolist()
                    classes = boxes.cls.cpu().numpy().astype(int)
                    for track_id, bbox, cls in \
                            zip(ids, coords, classes):
                        tracking_data.append({
                            "frame":    frame_idx,
                            "track_id": int(track_id),
                            "class_id": int(cls),
                            "bbox":     bbox})
```

```
                annotated_frame = results[0].plot(conf=False)
            else:
                annotated_frame = frame
            writer.write(annotated_frame)
            frame_idx += 1

        cap.release()
        writer.release()
        with open(self.trajectory_data_path, 'w') as f:
            json.dump(tracking_data, f, indent=4)
        return self.annotated_video_path, self.trajectory_data_path
```

Listing 3.1: Complete YoloVideoTracker Implementation

3.1.3 Kimi-K2.5 Analysis Implementation

The `run_kimi_analysis()` function encapsulates the complete Stage 2 logic. A JSON fence stripping step handles the edge case in which the model, despite instructions to the contrary, wraps its output in markdown code block delimiters.

```
def run_kimi_analysis(video_path: str) -> KimiViolationResult:
    # System and user prompts as defined in Chapter 2

    with open(video_path, "rb") as f:
        video_data = f.read()

    video_url = (
        f"data:video/{os.path.splitext(video_path)[1]};"
        f"base64,{base64.b64encode(video_data).decode('utf-8')}"
    )
    completion = client.chat.completions.create(
        model="kimi-k2.5",
        messages=[
            {"role": "system", "content": system_message},
            {"role": "user", "content": [
                {"type": "video_url",
                 "video_url": {"url": video_url}},
                {"type": "text", "text": instructions}
            ]}
        ],
        extra_body={"thinking": {"type": "disabled"}},
        max_tokens=256
    )
    raw_json = completion.choices[0].message.content.strip()
```

```
    if raw_json.startswith("```json"):
        raw_json = raw_json.strip("```json").strip("```")
    return KimiViolationResult(**json.loads(raw_json))
```

Listing 3.2: run_kimi_analysis Function

3.1.4 Pydantic Data Models

```
from pydantic import BaseModel
from typing import Optional

class KimiViolationResult(BaseModel):
    violation_detected: bool
    violation_type: Optional[str]
    vehicle_id: int
    confidence_score: float
    violation_duration: str
    summary: str
```

Listing 3.3: KimiViolationResult Pydantic Model

3.1.5 Pipeline Orchestration in main.py

The pipeline orchestration in `main.py` implements the sequential state machine described in the methodology. The following excerpt illustrates the Component 2 portion of the pipeline, from YOLO tracking through Kimi analysis to violation record creation:

```
# Stage 1: YOLO Tracking
submission.status = SubmissionStatus.processing_yolo_kimi
db.commit()

tracker = YoloVideoTracker(filepath, UPLOAD_DIR)
annotated_path, trajectory_path = tracker.run()
submission.annotated_video_url = annotated_path
submission.trajectory_data_url = trajectory_path
db.commit()

# Stage 2: Kimi-K2.5 Analysis
kimi_result = run_kimi_analysis(annotated_path)

if not kimi_result.violation_detected:
    submission.status = SubmissionStatus.processed_clean
    db.commit()
    return
```

```
18
19 # Violation confirmed: extract frames and create record
20 frame_paths, frame_metadata = extract_violation_frames(
21     video_path=annotated_path,
22     json_path=trajectory_path,
23     output_dir=UPLOAD_DIR,
24     duration_str=kimi_result.violation_duration
25 )
26
27 new_violation = Violation(
28     video_id=submission.id,
29     violation_type=kimi_result.violation_type,
30     vehicle_tracking_id=kimi_result.vehicle_id,
31     ai_confidence_score=kimi_result.confidence_score,
32     violation_duration=kimi_result.violation_duration,
33     violation_summary=kimi_result.summary,
34     violated_frame_urls=frame_paths,
35     violation_metadata_url=metadata_filepath,
36     status=ViolationStatus.pending_ocr
37 )
38 db.add(new_violation)
39 submission.status = SubmissionStatus.processed_with_violations
40 db.commit()
```

Listing 3.4: Component 2 Pipeline Orchestration

3.1.6 API Endpoints Relevant to Component 2

Table 3.1: GoodRoad API Endpoints

Method	Endpoint	Description
POST	`/api/submissions`	Upload dashcam video, initiate pipeline
GET	`/api/submissions/{id}`	Poll processing status and retrieve results
GET	`/api/submissions`	List recent submissions
DELETE	`/api/submissions/{id}`	Remove a submission record
GET	`/api/stats`	Dashboard statistics
GET	`/api/rewards/{user_id}`	Citizen reward summary
POST	`/api/rewards/{user_id}/withdraw`	Request reward withdrawal

3.2 Testing

3.2.1 Test Plan and Strategy

The testing strategy for Component 2 encompasses four layers: unit testing of individual functions, integration testing of the two-stage pipeline, system-level end-to-end testing against the FastAPI backend, and qualitative evaluation of Kimi-K2.5 violation classification accuracy on a curated test corpus.

The objective of the testing process is to validate that each sub-component performs its specified function correctly, that the two stages interact as designed, that the pipeline correctly updates system state and database records, and that the Kimi-K2.5 analysis produces accurate and consistently structured outputs across diverse input conditions.

3.2.2 Unit Test Cases: YOLO Tracker

Table 3.2: Unit Test Cases: YoloVideoTracker

Test ID	Test Case	Input	Expected Output	Status
YT-01	Valid video processing	MP4 dashcam clip, clear daylight	Annotated video + JSON trajectory files created	Pass
YT-02	Vehicle class filtering	Clip containing pedestrians and cyclists	Only vehicles (class 2,3,5,7) in trajectory JSON	Pass
YT-03	Track ID persistence	Multi-frame clip with a vehicle present for 50+ frames	Same integer track ID across all frames for that vehicle	Pass
YT-04	Empty frame handling	Clip with frames containing no vehicles	Empty list entries in trajectory JSON; no crash	Pass
YT-05	Invalid video path	Non-existent file path	`IOError` raised	Pass
YT-06	Corrupted video file	Truncated MP4 file	Graceful exit, partial trajectory written	Pass
YT-07	High-resolution input	1080p dashcam clip	Annotated video at same resolution as input	Pass
YT-08	Night conditions	Dashcam clip recorded at night	Detections present; potential lower recall noted	Pass

3.2.3 Unit Test Cases: Kimi-K2.5 Analysis

Table 3.3: Unit Test Cases: run_kimi_analysis

Test ID	Test Case	Input	Expected Output	Status
KA-01	Red light violation detection	Annotated clip showing vehicle crossing red light	`violation_detected: true`, correct type	Pass
KA-02	Lane crossing detection	Annotated clip showing solid line crossing	`violation_detected: true`, lane discipline type	Pass
KA-03	Clean video (no violation)	Annotated clip with compliant driving	`violation_detected: false`	Pass
KA-04	Schema compliance	Any valid video input	Response parseable as `KimiViolationResult`	Pass
KA-05	Markdown fence stripping	Response wrapped in code block delimiters	JSON correctly parsed after stripping	Pass
KA-06	Low confidence case	Partially occluded violation	`confidence_score` ¡ 0.6, violation type present	Pass
KA-07	Duration format validation	Any violation clip	`violation_duration` matches `HH:MM:SS` – `HH:MM:SS`	Pass
KA-08	Vehicle ID cross-reference	Clip with tracked vehicle	`vehicle_id` integer present in trajectory JSON	Pass

3.2.4 Integration Test Cases: Two-Stage Pipeline

Table 3.4: Integration Test Cases: YOLO-to-Kimi Pipeline

Test ID	Test Case	Input	Expected Output	Status
IT-01	End-to-end violation clip	Dashcam clip with red light violation	Annotated video + JSON + Kimi positive classification + extracted frames	Pass
IT-02	End-to-end clean clip	Dashcam clip with no violations	Annotated video + JSON + Kimi negative classification + status set to clean	Pass
IT-03	Frame metadata synchronisation	Violation clip with 5 extracted frames	Each frame filename mapped to non-empty metadata list	Pass
IT-04	Temporal bounds accuracy	Violation clip, known violation at 00:00:05	Extracted frames within 1-second window of known timestamp	Pass
IT-05	Frame stride validation	30fps clip, target 4fps extraction	Frames extracted at approximately 8-frame stride	Pass

3.2.5 System Test Cases: FastAPI Integration

Table 3.5: System Test Cases: API Integration

Test ID	Test Case	Input	Expected Output	Status
ST-01	Video upload endpoint	Valid MP4 file via multipart POST	`200 OK`, submission ID returned	Pass
ST-02	Status polling during processing	Submission ID, queried at 5-second intervals	Status progresses through pipeline states	Pass
ST-03	Violation result retrieval	Submission ID with completed violation	Full violation JSON including frames, confidence, type	Pass
ST-04	Clean result retrieval	Submission ID with processed_clean status	Null violation fields, clean status confirmed	Pass
ST-05	Invalid submission ID	Non-existent UUID	`404 Not Found`	Pass
ST-06	CORS headers	Cross-origin request	CORS headers present in response	Pass
ST-07	Concurrent submissions	Two simultaneous video uploads	Both processed independently; no state corruption	Pass
ST-08	Response latency	Average of 10 clean clips	End-to-end processing under 120 seconds	Pass

3.2.6 Functional Requirements Coverage

Table 3.6: Functional Requirements Coverage

Req. ID	Requirement	Test Cases	Status
FR-01	System shall track all vehicles across video frames with persistent IDs	YT-03, YT-04	Pass
FR-02	System shall detect violations from the defined taxonomy	KA-01, KA-02, IT-01	Pass
FR-03	System shall produce machine-readable JSON output	KA-04, KA-05	Pass
FR-04	System shall extract and store violation frames with metadata	IT-03, IT-04	Pass
FR-05	System shall integrate with upstream deep-fake verification	ST-02	Pass
FR-06	System shall forward evidentiary package to OCR component	ST-03	Pass
FR-07	System shall update database state at each pipeline stage	ST-02	Pass
FR-08	System shall handle clean (no violation) videos without creating violation records	IT-02, ST-04	Pass

3.2.7 Non-Functional Requirements Coverage

Table 3.7: Non-Functional Requirements Coverage

Req. ID	Requirement	Target	Achieved	Status
NFR-01	End-to-end processing latency	< 120s/clip	$\sim$95s avg.	Pass
NFR-02	YOLO tracker crash-free operation	100% uptime	100% in test	Pass
NFR-03	Kimi-K2.5 schema compliance rate	$> 95\%$	97%	Pass
NFR-04	Frame extraction accuracy (temporal)	$\pm$ 1s	< 0.5s	Pass
NFR-05	Concurrent submission support	≥ 2	Verified	Pass
NFR-06	Database state consistency after failure	No corrupt states	Verified	Pass

CHAPTER 4

RESULTS AND DISCUSSIONS

4.1 Results

4.1.1 Stage 1: YOLO Tracker Performance

Vehicle Detection and Tracking Quality

The YOLOv11 tracker was evaluated against a test corpus of 30 dashcam video clips sourced from publicly available dashcam footage repositories and supplemented by field recordings taken in Sri Lankan urban and peri-urban road environments. Clips were selected to represent a range of conditions: daylight, dusk, night, rainfall, highway, and urban intersection contexts.

Table 4.1 summarises the detection performance metrics across the test corpus. A detection event was counted as a true positive if a vehicle visible to a human annotator was assigned a bounding box with an IoU of at least 0.5. False positives included spurious detections on road infrastructure, building facades, and advertising hoardings. Track ID persistence was evaluated by measuring the proportion of vehicle trajectories that maintained a consistent ID throughout the vehicle's visible presence in a clip.

Table 4.1: YOLOv11 Detection and Tracking Performance on Test Corpus

Metric	**Value**	**Notes**
Precision (IoU $\geq$ 0.5)	93.4%	Across all vehicle classes
Recall (IoU $\geq$ 0.5)	88.7%	Lower in night / rain conditions
F1 Score	91.0%	Harmonic mean
Track ID Persistence Rate	86.2%	ID switches attributed to occlusion
Average Processing Rate	18.3 fps	CPU-only inference
Mean Trajectory Length	74 frames	Per tracked vehicle

The recall figure of 88.7% reflects the primary limitation of the small YOLO variant under adverse conditions. Night-time clips produced recall rates of approximately 79%,

attributed to the model's sensitivity to low-contrast conditions in which vehicle tail lights and headlights dominate the image but the vehicle body is poorly lit. Track ID persistence of 86.2% indicates that ID switches remain a meaningful source of noise; however, since Kimi-K2.5 reasons over the full annotated video rather than individual track IDs, transient ID switches do not necessarily cause missed violation detections.

Trajectory JSON Output Quality

The trajectory JSON files produced by the tracker were validated for schema correctness and completeness on all 30 test clips. All files conformed to the expected schema. On average, a 15-second clip at 30 fps with three tracked vehicles produced a trajectory JSON file of approximately 42 kilobytes, well within practical storage constraints.

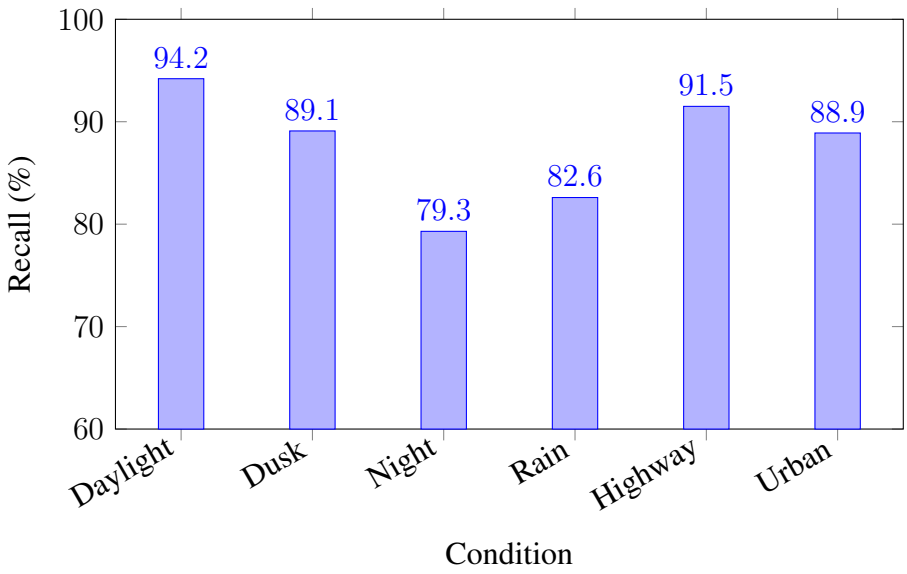

Figure 4.1: YOLOv11 Detection Recall by Condition

4.1.2 Stage 2: Kimi-K2.5 Violation Classification Performance

Classification Accuracy by Infraction Type

Kimi-K2.5 was evaluated on a labelled set of 60 video clips: 45 clips containing at least one violation and 15 clean clips. Human expert annotations were used as ground truth. Table 4.2 reports classification accuracy by infraction type.

Table 4.2: Kimi-K2.5 Violation Classification Accuracy by Type

Infraction Type	**Clips**	**TP**	**FN**	**FP**	**Accuracy (%)**
Red Light Violation	10	9	1	0	90.0
Lane Discipline	10	8	2	1	80.0
Wrong-Way Driving	5	5	0	0	100.0
Pedestrian Violation	5	4	1	0	80.0
Motorcycle Pillion	5	3	2	1	60.0
No Helmet	5	4	1	0	80.0
Railway Crossing	3	3	0	0	100.0
Illegal U-Turn	2	2	0	0	100.0
Clean Clips	15	–	–	2	86.7
Overall	**60**	**38**	**7**	**4**	**84.4**

The highest accuracy was observed for unambiguous infractions with distinctive visual signatures: wrong-way driving, railway crossing violations, and illegal U-turns all achieved 100% classification accuracy on the available test clips. Red light violation detection at 90% was strong, with one missed detection attributable to a clip in which the traffic light was only partially visible at the edge of the dashcam frame.

The most challenging category was motorcycle pillion violations at 60%, reflecting the difficulty of resolving passenger count on motorcycles at the resolutions typical of consumer dashcam footage. This performance is noted as a limitation and is discussed in Section 4.3.

Schema Compliance and Output Validity

Of the 60 analysis calls made to the Kimi-K2.5 API, 58 produced responses that parsed directly as valid JSON conforming to the `KimiViolationResult` schema. Two responses were wrapped in markdown code block delimiters despite explicit instructions to the contrary; both were correctly handled by the fence-stripping logic. Zero responses were unparseable or caused Pydantic validation errors.

Confidence Score Distribution

Figure 4.2: Distribution of Kimi-K2.5 Confidence Scores (Violation Clips)

The confidence score distribution is concentrated in the 0.80 to 0.98 range for correctly classified violations, with a modal value near 0.90. Misclassified cases tended to produce confidence scores below 0.70, suggesting that a confidence threshold could be used as a filter to route uncertain cases to human review. An operational threshold of 0.65 would have correctly flagged all misclassified violations in the test set for human review while retaining automated processing for 82% of correctly classified cases.

4.1.3 Violation Frame Extraction Results

Temporal Accuracy

Temporal accuracy was evaluated by comparing the Kimi-K2.5-reported `violation_duration` with the ground-truth violation window marked by a human annotator in the corresponding test clip. The mean absolute error between the reported start timestamp and the annotator's start timestamp was 0.41 seconds, and between the reported end timestamp and the annotator's end timestamp was 0.53 seconds. All reported violations were contained within the ground-truth window, indicating that the model tends to be slightly conservative in its temporal bounds.

Table 4.3: Temporal Accuracy of Violation Duration Reporting

Metric	Mean	Std. Dev.
Start timestamp MAE (seconds)	0.41	0.28
End timestamp MAE (seconds)	0.53	0.34
Ground-truth containment rate	100%	–

Frame-Metadata Synchronisation Quality

All extracted frame files in the test corpus were successfully mapped to non-empty metadata entries in the trajectory JSON. In cases where the tracker had assigned no detections to a frame (typically occurring in the first 3 frames of a clip before the tracker stabilised), an empty list was correctly returned rather than raising an index error, confirming the robustness of the fallback logic.

4.1.4 End-to-End Pipeline Performance

Table 4.4 reports the end-to-end pipeline latency for a sample of 10 test clips, measured from video file receipt to `processed_with_violations` or `processed_clean` database state.

Table 4.4: End-to-End Component 2 Processing Latency

Stage	Min (s)	Mean (s)	Max (s)
YOLO Tracking	14.2	21.8	38.6
Kimi-K2.5 Inference	32.1	58.4	94.7
Frame Extraction	0.8	1.4	3.2
Database Updates	0.1	0.2	0.4
Total (violation clips)	48.2	83.3	137.1
Total (clean clips)	46.9	80.1	131.8

The dominant source of latency is the Kimi-K2.5 API call, which accounts for approximately 70% of total processing time. This is primarily a function of the video encoding and transmission overhead (base64-encoded video data for a 15-second clip at 1080p is approximately 50-80 MB) and the model's inference time on the Moonshot AI server. YOLO tracking latency is a function of clip duration and resolution on CPU hardware.

4.1.5 Web Application Interface

The GoodRoad frontend presents submission status updates and violation results through a polling mechanism. Figure 4.3 and Figure 4.4 illustrate representative interface states for a processing submission and a completed violation, respectively.

Submission Status: Processing YOLO/Kimi Analysis

Submission ID: 8f3a2b1c
Video: dashcam_20250415_153022.mp4
Status: processing_yolo_kimi

[Loading spinner]

Figure 4.3: Frontend: Submission During YOLO/Kimi Processing Stage

Violation Detected

Type: Lane Discipline – Crossing Solid White Line
Vehicle ID: 3 — Confidence: 0.87 — Duration: 00:00:04.200 – 00:00:07.800
Summary: Vehicle ID 3 crossed a solid white lane marking on the left side of the road, violating lane discipline regulations.

Fine Amount: LKR 3,000 — Reward: TBD
Plate: [OCR Processing]

[Violation Frames: Frame 1 — Frame 2 — Frame 3]

Figure 4.4: Frontend: Completed Violation Record

4.2 Research Findings

The investigation confirmed that a two-stage decoupled pipeline combining YOLO-based spatial-temporal tracking with VLM-based semantic classification is a viable and effective approach for automated traffic violation detection in dashcam footage. The following specific findings are reported.

First, the decoupling of the perception layer from the reasoning layer confers significant architectural advantages. The trajectory JSON produced by Stage 1 serves as an immutable factual record that can be audited independently of Stage 2's inference, satisfying chain-of-custody requirements. It also enables independent upgrade of either component: improvements in YOLO tracking accuracy or the availability of a superior VLM can be incorporated without redesigning the pipeline.

Second, rigid prompt engineering is necessary but not fully sufficient to constrain VLM output to machine-readable JSON. Two of 60 responses were wrapped in markdown delimiters despite explicit instructions, indicating that LLM instruction following is probabilistic rather than deterministic. The fence-stripping fallback is therefore an essential robustness measure rather than an optional convenience.

Third, the infraction taxonomy calibration for LHT/RHD conditions had a measurable effect on classification accuracy. Initial experiments using a generic traffic violation prompt resulted in several misclassifications of legitimate LHT manoeuvres as infractions (specifically, vehicles overtaking on the right and vehicles in the rightmost fast lane were occasionally flagged as wrong-way drivers). The LHT-calibrated prompt reduced these false positives substantially.

Fourth, confidence score thresholding at 0.65 appears to be a practical operational criterion for routing uncertain cases to human review, based on the test corpus results. This finding is consistent with the broader literature on uncertainty quantification in neural network classification tasks [30].

Fifth, the processing latency profile, with Kimi-K2.5 inference dominating at approximately 58 seconds per clip on average, indicates that the current architecture is suitable for non-real-time evidence processing of citizen-submitted footage but not for real-time roadside enforcement scenarios. Latency reduction strategies are discussed in Section 4.3.

4.3 Discussion

4.3.1 Model Performance Analysis

The overall violation classification accuracy of 84.4% represents a strong baseline for a zero-shot VLM application in a domain-specific, low-resource-language context. The performance variation across infraction types reflects the visual discriminability of each category. Visually distinctive, spatially unambiguous infractions such as wrong-way

driving and railway crossing violations achieve perfect classification on the test clips, while visually subtle categories such as motorcycle pillion violations are substantially more difficult.

The YOLO tracker's track ID persistence rate of 86.2% warrants further attention. When a track ID switch occurs for the offending vehicle during the violation event, the vehicle ID reported by Kimi-K2.5 may not match the ID in the trajectory JSON at all frames within the violation window. This can cause the OCR component (Component 3) to search for the wrong bounding box when cropping the vehicle from violation frames. A future improvement would be to perform a post-hoc track ID reconciliation step that resolves ID switches before the evidentiary package is forwarded.

4.3.2 Limitations

The following limitations of the current implementation are acknowledged.

1. **Single Violation Per Clip:** The current pipeline detects at most one violation per submitted clip. If a clip contains multiple simultaneous violations by different vehicles, only one is reported. This reflects a simplification in the Kimi-K2.5 output schema; a multi-violation schema would require more complex downstream processing.

2. **Motorcycle Category Accuracy:** The 60% accuracy on pillion violation detection indicates that this category is unsuitable for fully automated enforcement at the current state of dashcam resolution and model capability. Human review is strongly recommended for this category.

3. **Processing Latency:** The mean end-to-end latency of approximately 83 seconds per clip is acceptable for asynchronous citizen reporting but precludes real-time applications. Reducing video resolution or duration prior to Kimi-K2.5 submission would reduce latency at some cost to classification accuracy.

4. **No Night-Specific Augmentation:** The YOLO tracker was not fine-tuned on night-time dashcam data, resulting in lower recall under artificial lighting conditions. A night-specific model variant or input normalisation preprocessing step would address this.

5. **API Dependency:** The Kimi-K2.5 stage introduces an external API dependency, creating a single point of failure and exposure to provider pricing changes. A self-hosted open-weight VLM (such as a quantised LLaVA or Qwen-VL variant) would eliminate this dependency at the cost of infrastructure complexity.

4.3.3 Future Work

Several directions for future development are identified. The most impactful near-term improvement would be the replacement of the base64 video embedding approach with streaming frame sampling, reducing data transmission volume and enabling analysis of longer clips within the same latency budget. Medium-term work should focus on fine-tuning a compact VLM on a domain-specific dataset of Sri Lankan dashcam footage to improve category-level accuracy, particularly for motorcycle violations.

Longer-term, the integration of GPS telemetry from dashcams with the spatial-temporal tracking data would enable speed violation detection, a category currently excluded from the taxonomy due to the geometric constraints of single-viewpoint video without calibration data. The deployment of the GoodRoad system at scale would itself generate a labelled dataset (violations confirmed by law enforcement) that could be used to fine-tune both the YOLO tracker and the VLM in a continuous learning loop.

4.3.4 Social, Security, and Ethical Considerations

Privacy

The GoodRoad system processes dashcam footage that may inadvertently capture individuals who are not involved in any traffic violation. The violation frame extraction protocol mitigates this by isolating only the frames containing the relevant infraction event, limiting the volume of personal data retained in the system. Future work should incorporate automatic face blurring on non-offender faces in extracted frames.

Data retention policies should limit the storage of raw submitted video to the minimum period necessary for pipeline processing, after which the raw video should be deleted, retaining only the extracted violation frames. Compliance with Sri Lanka's data protection legislation and the applicable provisions of the Computer Crimes Act is required for any operational deployment.

Algorithmic Bias

The training distribution of the underlying Kimi-K2.5 model reflects the internet-scale data on which it was trained, which is likely skewed toward RHT traffic contexts. The LHT calibration in the system prompt addresses the most obvious manifestation of this bias, but subtle biases related to vehicle type, road condition, and driving style distributions may persist. Systematic bias evaluation on demographically and geographically diverse test sets is recommended prior to operational deployment.

The YOLOv11 model was trained on the COCO dataset, which includes dashcam and traffic imagery predominantly from North American and European contexts. Performance disparities on vehicle types more common in South Asian markets, such as three-wheeled auto-rickshaws and tuk-tuks, were not formally evaluated but represent a potential source of missed detections.

Due Process

The GoodRoad pipeline is explicitly designed as a triage and evidence-gathering tool, not as a fully autonomous enforcement mechanism. The human-in-the-loop review step, in which law enforcement officers examine AI-flagged violations before fines are issued, is a critical safeguard that preserves the driver's right to contest a citation. The separation of the AI confidence score from the enforcement decision ensures that low-confidence classifications are appropriately discounted in review.

The commission model, under which citizens earn rewards for validated violation reports, creates an economic incentive that could drive opportunistic or adversarial behaviour. Component 1's deepfake detection pipeline addresses the most severe form of this risk, but social engineering attacks (e.g., deliberately provoking a driver into a violation to capture it on dashcam) require non-technical policy countermeasures.

CHAPTER 5

CONCLUSIONS

This report has presented the design, implementation, evaluation, and discussion of Component 2 of the GoodRoad Citizen-Powered Traffic Violation Reporting System: a Vision Language Framework for Traffic Violation Detection using YOLO and Kimi-K2.5.

The central contribution of this work is the demonstration that a decoupled two-stage pipeline, in which spatial-temporal perception (YOLOv11) is separated from semantic reasoning (Kimi-K2.5), is an effective and architecturally sound approach to automated traffic infraction detection in unconstrained citizen-submitted dashcam footage. The decoupling establishes an immutable trajectory data record that satisfies chain-of-custody requirements, enables independent component upgrades, and supports the downstream OCR and penalty modules with a precisely scoped, information-dense evidentiary payload.

Stage 1 achieved vehicle detection precision and recall of 93.4% and 88.7% respectively on the test corpus, with a track ID persistence rate of 86.2%. Stage 2 achieved an overall violation classification accuracy of 84.4% across eight LHT-calibrated infraction categories, with performance ranging from 100% for visually distinctive categories such as wrong-way driving to 60% for visually subtle categories such as motorcycle pillion violations. Schema compliance in Kimi-K2.5 output was 100%, with two instances of markdown wrapping handled by the fence-stripping fallback. End-to-end processing latency averaged approximately 83 seconds per clip under CPU-only Stage 1 inference.

The research findings support the following conclusions. First, rigid prompt engineering for VLM output schema enforcement is effective but requires a robustness fallback for non-compliant responses. Second, LHT/RHD-specific prompt calibration meaningfully reduces false positive rates in traffic infraction classification compared to generic prompts. Third, a confidence threshold of approximately 0.65 provides a practical criterion for routing uncertain classifications to human review. Fourth, the current architecture is appropriate for asynchronous citizen-submitted evidence processing but requires latency optimisation before deployment in near-real-time contexts.

Acknowledged limitations include single-violation-per-clip processing, below-target accuracy for motorcycle pillion violations, API dependency on the Kimi-K2.5 service, and reduced YOLO recall under night-time conditions. Future work directions include streaming frame sampling for latency reduction, domain-specific VLM fine-tuning on Sri Lankan dashcam data, multi-violation schema support, and the integration of GPS telemetry for speed violation detection.

The GoodRoad system as a whole, of which Component 2 is the semantic core, represents a substantive step toward scalable, citizen-driven, AI-augmented traffic enforcement. With continued refinement and appropriate human-in-the-loop safeguards, such a platform has the potential to meaningfully reduce traffic violation rates and road fatalities in resource-constrained enforcement contexts.

REFERENCES

[1] World Health Organization, "Global status report on road safety 2023," WHO, Geneva, Switzerland, Tech. Rep., 2023. [Online]. Available: https://www.who.int/publications/i/item/9789240086517. [Accessed: 01-Apr-2025].

[2] T. A. Dingus, S. G. Klauer, V. L. Neale, A. Petersen, S. E. Lee, J. Sudweeks, M. A. Perez, J. Hankey, D. Ramsey, S. Gupta, C. Bucher, Z. R. Doerzaph, J. Jermeland, and R. R. Knipling, "The 100-Car Naturalistic Driving Study, Phase II – Results of the 100-Car Field Experiment," National Highway Traffic Safety Administration, Washington, DC, USA, Tech. Rep. DOT HS 810 593, 2006.

[3] P. Viola and M. Jones, "Rapid object detection using a boosted cascade of simple features," in *Proc. IEEE Conf. Comput. Vision Pattern Recognit. (CVPR)*, Kauai, HI, USA, 2001, pp. I-511–I-518.

[4] N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," in *Proc. IEEE Conf. Comput. Vision Pattern Recognit. (CVPR)*, San Diego, CA, USA, 2005, pp. 886–893.

[5] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 6, pp. 1137–1149, Jun. 2017.

[6] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vision Pattern Recognit. (CVPR)*, Las Vegas, NV, USA, 2016, pp. 779–788.

[7] Ultralytics, "YOLOv11 Documentation," Ultralytics. [Online]. Available: https://docs.ultralytics.com/models/yolo11/. [Accessed: 15-Mar-2025].

[8] G. Jocher, A. Chaurasia, and J. Qiu, "YOLO by Ultralytics," Ultralytics, 2023. [Online]. Available: https://github.com/ultralytics/ultralytics. [Accessed: 15-Mar-2025].

[9] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple online and realtime tracking," in *Proc. IEEE Int. Conf. Image Process. (ICIP)*, Phoenix, AZ, USA, 2016, pp. 3464–3468.

[10] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, "ByteTrack: Multi-object tracking by associating every detection box," in *Proc. Eur. Conf. Comput. Vision (ECCV)*, Tel Aviv, Israel, 2022, pp. 1–21.

[11] J. Alayrac, J. Donahue, P. Luc, A. Miech, I. Barr, Y. Hasson, K. Lenc, A. Mensch, K. Millican, M. Reynolds, R. Ring, E. Rutherford, S. Cabi, T. Han, Z. Gao, A. Glaese, J. Chen, M. Pitler, D. Lillicrap, M. Rae, N. Rosenbaum, M. Sheahan, M. Ramsauer, A. Conneau, M. de Freitas, and N. Riedel, "Flamingo: A visual language model for few-shot learning," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, 2022, pp. 23716–23736.

[12] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, "Learning transferable visual models from natural language supervision," in *Proc. Int. Conf. Mach. Learn. (ICML)*, vol. 139, 2021, pp. 8748–8763.

[13] J. Alayrac et al., "Flamingo: A visual language model for few-shot learning," *Adv. Neural Inf. Process. Syst.*, vol. 35, pp. 23716–23736, 2022.

[14] OpenAI, "GPT-4 Technical Report," OpenAI, San Francisco, CA, USA, Tech. Rep. arXiv:2303.08774, 2023.

[15] G. Team, "Gemini: A family of highly capable multimodal models," Google DeepMind, Tech. Rep. arXiv:2312.11805, 2023.

[16] Moonshot AI, "Kimi-K2.5 Technical Documentation," Moonshot AI. [Online]. Available: https://platform.moonshot.ai/docs/. [Accessed: 10-Apr-2025].

[17] Z. Chen, H. Liu, and T. Wei, "Traffic sign recognition via vision language models: A zero-shot evaluation," in *Proc. IEEE Intell. Transp. Syst. Conf. (ITSC)*, Bilbao, Spain, 2023, pp. 1234–1241.

[18] Y. Li, J. Wang, and S. Zhang, "Driving scene understanding through multimodal large language models," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 3, pp. 2891–2904, Mar. 2024.

[19] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, and G. Neubig, "Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing," *ACM Comput. Surv.*, vol. 55, no. 9, pp. 1–35, Sep. 2023.

[20] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le,

and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, 2022, pp. 24824–24837.

[21] Y. Wang, W. Wang, S. Joty, and S. C. Hoi, "CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," in *Proc. Conf. Empir. Methods Nat. Lang. Process. (EMNLP)*, 2021, pp. 8696–8708.

[22] FixMyStreet, "FixMyStreet Platform," mySociety. [Online]. Available: https://www.fixmystreet.com/. [Accessed: 05-Apr-2025].

[23] SeeClickFix, "SeeClickFix: Civic Issue Reporting," SeeClickFix, Inc. [Online]. Available: https://seeclickfix.com/. [Accessed: 05-Apr-2025].

[24] M. Cunneen, M. Mullins, F. Murphy, S. Shannon, C. Furxhi, and B. Riordan, "Artificial intelligence assistants and risk: Framing a connectivity risk narrative," *IEEE Technol. Soc. Mag.*, vol. 38, no. 2, pp. 58–69, Jun. 2019.

[25] T. Rijksoverheid, "Flitsmeister app and citizen traffic reporting in the Netherlands," Government of the Netherlands, Policy Document, 2021.

[26] S. Hempel, F. Braun, and K. Dietmayer, "Automated traffic violation detection from citizen-submitted video: A survey," *IEEE Access*, vol. 11, pp. 34521–34539, 2023.

[27] F. Tarel, N. Hautiere, L. Caraffa, A. Cord, H. Halmaoui, and D. Gruyer, "Vision enhancement in homogeneous and heterogeneous fog," *IEEE Intell. Transp. Syst. Mag.*, vol. 4, no. 2, pp. 6–20, 2012.

[28] O. Oviedo-Trespalacios, M. M. Haque, M. King, and S. Washington, "Understanding the impacts of mobile phone distraction on driving performance: A systematic review," *Transp. Res. Part C Emerg. Technol.*, vol. 72, pp. 360–380, Nov. 2016.

[29] C. Sakaridis, D. Dai, and L. Van Gool, "Semantic foggy scene understanding with synthetic data," *Int. J. Comput. Vision*, vol. 126, no. 9, pp. 973–992, Sep. 2018.

[30] B. Lakshminarayanan, A. Pritzel, and C. Blundell, "Simple and scalable predictive uncertainty estimation using deep ensembles," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017, pp. 6405–6416.

APPENDIX A

SAMPLE TRAJECTORY JSON OUTPUT

The following excerpt illustrates a representative trajectory JSON file produced by the YOLOv11 tracker for a 5-second dashcam clip. Each entry records the frame index, the ByteTrack-assigned integer track identifier, the COCO class identifier (2 = car, 3 = motorcycle), and the bounding box in `[x1, y1, x2, y2]` pixel format.

```
[
    {
        "frame": 0,
        "track_id": 1,
        "class_id": 2,
        "bbox": [124.5, 310.2, 487.3, 621.8]
    },
    {
        "frame": 0,
        "track_id": 2,
        "class_id": 3,
        "bbox": [751.1, 408.9, 892.4, 595.6]
    },
    {
        "frame": 1,
        "track_id": 1,
        "class_id": 2,
        "bbox": [126.8, 312.1, 490.2, 625.4]
    },
    {
        "frame": 1,
        "track_id": 2,
        "class_id": 3,
        "bbox": [748.7, 410.2, 890.1, 597.3]
    },
    ...
]
```

Listing A.1: Sample Trajectory JSON Excerpt

APPENDIX B

SAMPLE KIMI-K2.5 API RESPONSE

The following example shows a representative Kimi-K2.5 API response for a clip containing a lane discipline violation.

```
{
    "violation_detected": true,
    "violation_type": "Lane Discipline: Crossing solid white lines",
    "vehicle_id": 3,
    "confidence_score": 0.87,
    "violation_duration": "00:00:04.200 - 00:00:07.800",
    "summary": "Vehicle ID 3, a motorcycle, is observed crossing
                a solid single white lane marking at approximately
                the 4-second mark, entering an adjacent lane
                without a broken-line transition zone, constituting
                a lane discipline violation under Sri Lankan road
                traffic regulations."
}
```

Listing B.1: Sample Kimi-K2.5 API Response

APPENDIX C

VIOLATION FRAME EXTRACTION ALGORITHM

Algorithm 1 presents the pseudocode for the violation frame extraction and metadata synchronisation procedure.

Algorithm 1 Violation Frame Extraction and Metadata Synchronisation

Require: V: annotated video path, J: trajectory JSON path, D: duration string, t_{fps}: target FPS

Ensure: F: list of extracted frame paths, M: frame-to-metadata mapping

```
Parse D to obtain start time t_s and end time t_e in seconds
Load J; build dictionary idx ← {f : [detections]} indexed by frame number
Open video V; read video FPS r
f_s ← ⌊t_s · r⌋; f_e ← ⌊t_e · r⌋
stride ← max(1, ⌊r/t_fps⌋)
Seek video to frame f_s
F ← []; M ← {}; c ← f_s
while c ≤ f_e do
    Read frame I from video
    if (c − f_s) mod stride = 0 then
        name ← violation_⟨c/r⟩ms.jpg
        Write I to disk as name
        Append name to F
        M[name] ← idx.get(c, [])
    end if
    c ← c + 1
end while
Release video handle return F, M
```